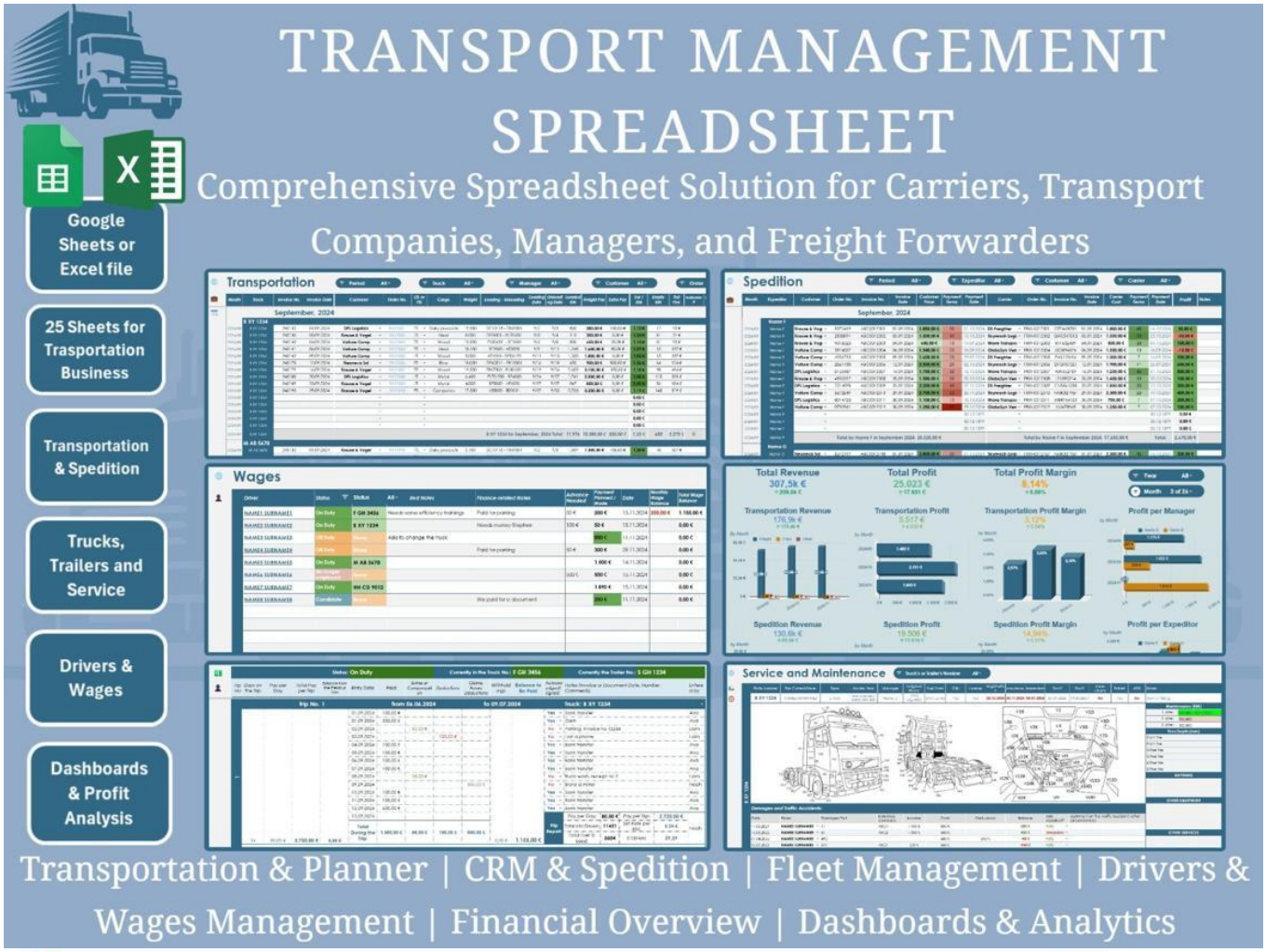
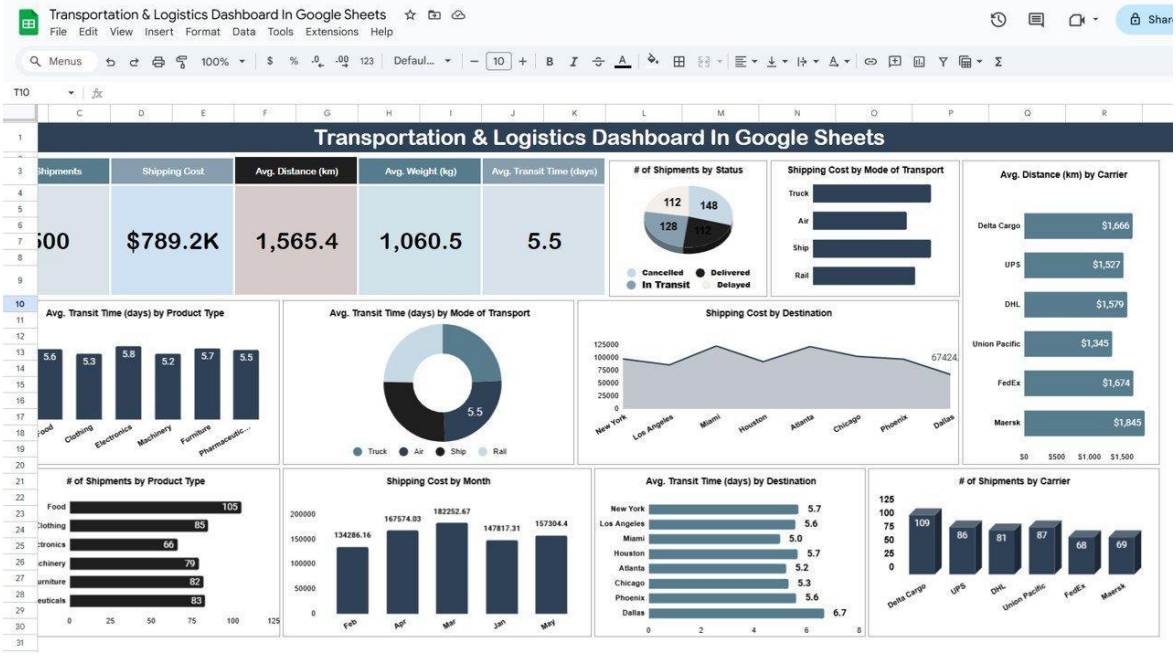
Transport Data Import Management System







1. Project overview

Project title: Transport Data Import Management System Using Spreadsheets

Objective: To develop a web application that imports transport records from Excel or CSV spreadsheets, validates the data, and stores the correct records in a database.

Example: A transport company maintains vehicle details, driver information, routes, and trip records in an Excel file. Instead of entering each record manually, the user uploads the spreadsheet, and the system imports the data automatically.

2. Technologies used

- Frontend: HTML, CSS, Bootstrap
- Backend: Python Flask
- Database: SQLite
- Data import: Pandas and OpenPyXL

- File formats: CSV and Excel (`.xlsx`)

3. Main modules

1. Login page – user login.
2. Dashboard – total vehicles, drivers, routes, and imported records.
3. Spreadsheet upload – upload an Excel or CSV file.
4. Data validation – identify missing or incorrect information.
5. Data transformation – standardize vehicle numbers, names, and route information.
6. Database import – save valid records into SQLite.
7. View records – display imported transport data.
8. Import report – show successful and failed records.

4. Project folder structure

Create a folder named `TransportDataImport` in VS Code.

TransportDataImport/

`app.py` — Main application

`requirements.txt` — Required libraries

`templates/`

`login.html`

`dashboard.html`

`upload.html`

`records.html`

`transport.db` — Created automatically

5. Install the required libraries

Open the VS Code terminal and run:

```
pip install flask pandas openpyxl
```

Create `requirements.txt`:

```
Flask
pandas
openpyxl
```

6. Main application code — `app.py`

This version provides a working upload, validation, import, and record-viewing system. It uses a simple demonstration login; add proper authentication before using it beyond a college demo.

```
from flask import (
Flask, render_template, request,
redirect, url_for, flash, session
)
import pandas as pd
import sqlite3
import os
import re
from functools import wraps

app = Flask(__name__)
app.secret_key = os.environ.get(
"FLASK_SECRET_KEY", "college-demo-change-this"
)

DB = "transport.db"
UPLOAD_FOLDER = "uploads"
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

def connect_db():
con = sqlite3.connect(DB)
con.row_factory = sqlite3.Row
return con

def init_db():
with connect_db() as con:
con.execute("""
CREATE TABLE IF NOT EXISTS transport (
id INTEGER PRIMARY KEY AUTOINCREMENT,
vehicle_no TEXT NOT NULL,
driver_name TEXT NOT NULL,
source TEXT NOT NULL,
destination TEXT NOT NULL,
distance REAL NOT NULL,
import_date TEXT DEFAULT CURRENT_TIMESTAMP,
UNIQUE(vehicle_no, source, destination)
)
""")

def login_required(func):
@wraps(func)
def wrapper(*args, **kwargs):
if not session.get("logged_in"):
return redirect(url_for("login"))
return func(*args, **kwargs)
return wrapper

@app.route("/", methods=["GET", "POST"])
def login():
if request.method == "POST":
# Demonstration credentials only
if (request.form.get("username") == "admin"
and request.form.get("password") == "admin123"):
session["logged_in"] = True
return redirect(url_for("dashboard"))
```

```
flash("Invalid username or password")

return render_template("login.html")

@app.route("/logout")
def logout():
session.clear()
return redirect(url_for("login"))

@app.route("/dashboard")
@login_required
def dashboard():
with connect_db() as con:
total = con.execute(
"SELECT COUNT(*) FROM transport"
).fetchone()[0]
return render_template("dashboard.html", total=total)

@app.route("/upload", methods=["GET", "POST"])
@login_required
def upload():
if request.method == "POST":
file = request.files.get("file")

if not file or not file.filename:
flash("Please select a spreadsheet")
return redirect(url_for("upload"))

ext = os.path.splitext(file.filename)[1].lower()
if ext not in [".csv", ".xlsx"]:
flash("Upload only CSV or XLSX files")
return redirect(url_for("upload"))

try:
if ext == ".csv":
df = pd.read_csv(file)
else:
df = pd.read_excel(file, engine="openpyxl")

df.columns = [
str(c).strip().lower().replace(" ", "_")
for c in df.columns
]

required = [
"vehicle_no", "driver_name",
"source", "destination", "distance"
]

if not all(c in df.columns for c in required):
flash("Required columns: " + ", ".join(required))
return redirect(url_for("upload"))

df = df[required].copy()
df["distance"] = pd.to_numeric(
df["distance"], errors="coerce"
)

valid = (
df[required[:4]].notna().all(axis=1)
& df[required[:4]].apply(
```

```
lambda col: col.astype(str).str.strip().ne("")
).all(axis=1)
& df["distance"].notna()
& (df["distance"] > 0)
)

df = df.loc[valid].copy()
df["vehicle_no"] = (
df["vehicle_no"].astype(str).str.strip().str.upper()
)

imported = 0
skipped = int((~valid).sum())

with connect_db() as con:
for row in df.to_dict("records"):
cur = con.execute("""
INSERT OR IGNORE INTO transport
(vehicle_no, driver_name, source,
destination, distance)
VALUES (?, ?, ?, ?, ?)
""", (
row["vehicle_no"],
str(row["driver_name"]).strip(),
str(row["source"]).strip(),
str(row["destination"]).strip(),
float(row["distance"])
))
if cur.rowcount == 1:
imported += 1
else:
skipped += 1

flash(
f"Import complete! Added: {imported}; "
f"invalid or duplicate rows skipped: {skipped}"
)

except Exception as e:
flash(f"Could not import file: {e}")

return redirect(url_for("records"))

return render_template("upload.html")

@app.route("/records")
@login_required
def records():
with connect_db() as con:
rows = con.execute(
"SELECT * FROM transport ORDER BY id DESC"
).fetchall()
return render_template("records.html", rows=rows)

if __name__ == "__main__":
init_db()
app.run(debug=True)
```

7. Create the HTML pages

Create the `templates` folder and add the following four files.

login.html

```
<!DOCTYPE html>
<html>
<head>
<title>Transport Login</title>
</head>
<body>
<h1>Transport Data Import System</h1>
{% for message in get_flashed_messages() %}
<p>{{ message }}</p>
{% endfor %}
<form method="POST">
<input name="username" placeholder="Username" required>
<input name="password" type="password"
placeholder="Password" required>
<button type="submit">Login</button>
</form>
<p>Demo username: admin</p>
<p>Demo password: admin123</p>
</body>
</html>
```

dashboard.html

```
<!DOCTYPE html>
<html>
<head><title>Dashboard</title></head>
<body>
<h1>Transport Management Dashboard</h1>
<h2>Total Imported Records: {{ total }}</h2>
<a href="{{ url_for('upload') }}">Import Spreadsheet</a>
<br><br>
<a href="{{ url_for('records') }}">View Transport Records</a>
<br><br>
<a href="{{ url_for('logout') }}">Logout</a>
</body>
</html>
```

upload.html

```
<!DOCTYPE html>
<html>
<head><title>Import Data</title></head>
<body>
<h1>Import Transport Spreadsheet</h1>

{% for message in get_flashed_messages() %}
<p>{{ message }}</p>
{% endfor %}

<p>Accepted formats: CSV and XLSX</p>

<p>Required columns:</p>
```

```
<code>
vehicle_no, driver_name, source,
destination, distance
</code>

<form method="POST" enctype="multipart/form-data">
<input type="file" name="file"
accept=".csv,.xlsx" required>
<button type="submit">Import Data</button>
</form>

<a href="{{ url_for('dashboard') }}">Back to Dashboard</a>
</body>
</html>
```

records.html

```
<!DOCTYPE html>
<html>
<head><title>Transport Records</title></head>
<body>
<h1>Imported Transport Records</h1>

{% for message in get_flashed_messages() %}
<p>{{ message }}</p>
{% endfor %}

<table border="1" cellpadding="8">
<tr>
<th>ID</th>
<th>Vehicle Number</th>
<th>Driver Name</th>
<th>Source</th>
<th>Destination</th>
<th>Distance (km)</th>
</tr>
{% for r in rows %}
<tr>
<td>{{ r.id }}</td>
<td>{{ r.vehicle_no }}</td>
<td>{{ r.driver_name }}</td>
<td>{{ r.source }}</td>
<td>{{ r.destination }}</td>
<td>{{ r.distance }}</td>
</tr>
{% endfor %}
</table>

<p><a href="{{ url_for('upload') }}">Import More Data</a></p>
<p><a href="{{ url_for('dashboard') }}">Dashboard</a></p>
</body>
</html>
```

8. Sample spreadsheet data

Create an Excel spreadsheet with these exact column names and sample data.

vehicle_no	driver_name	source	destination	distance
TN67AB1234	Ravi	Virudhunagar	Madurai	45
TN67CD5678	Kumar	Sivakasi	Madurai	65
TN67EF9012	Arun	Sattur	Kovilpatti	30
TN67GH3456	Mani	Madurai	Chennai	460

Save the file as `transport_data.xlsx`. Upload it through the application to test the import feature.

9. How to run the project

1. Open the project folder in VS Code.
2. Install the required libraries.
3. Create the Python file and four HTML files shown above.
4. Open the terminal in the project folder.
5. Run:

```
python app.py
```

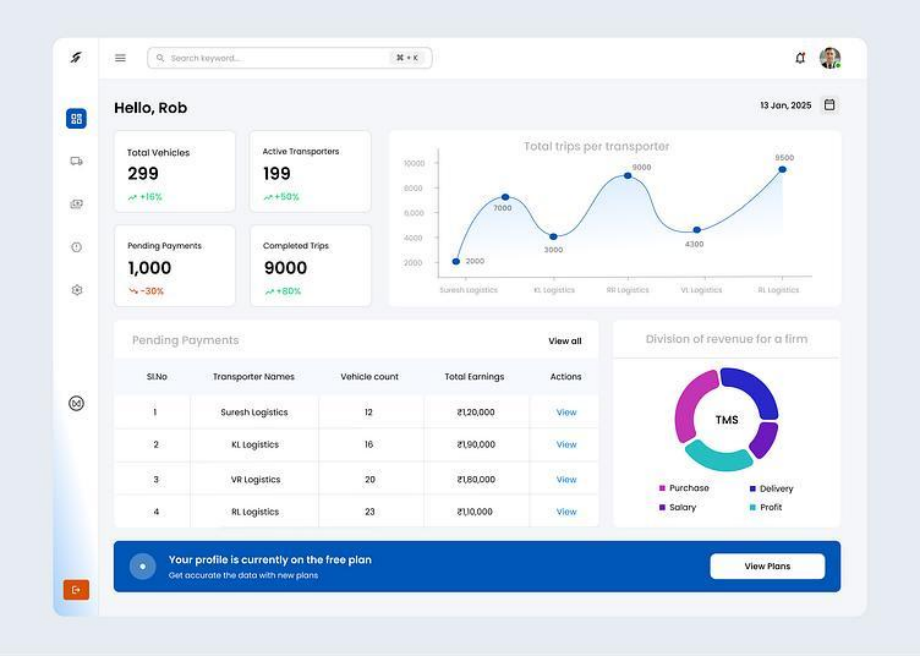
6. Open the local address displayed in the terminal, usually `http://127.0.0.1:5000`.
7. Log in with the demonstration credentials.
8. Open Import Spreadsheet, upload the sample Excel file, and view the imported records.

10. Expected output screenshots



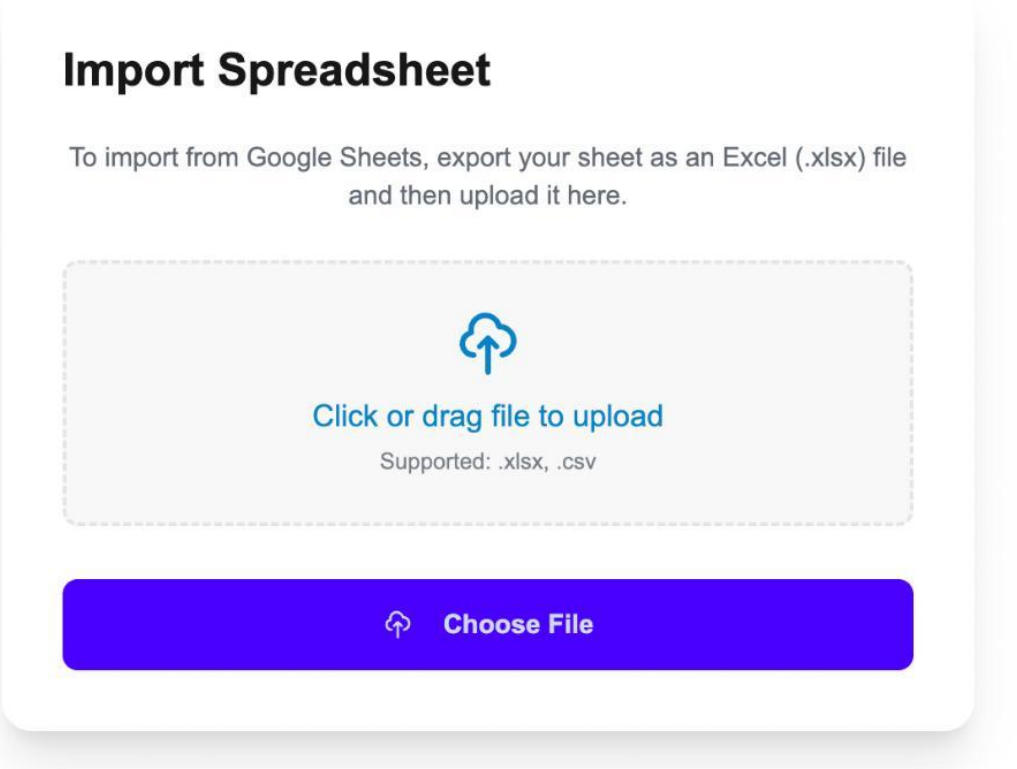
Screenshot 1 — Login page

Username and password fields with a login button.



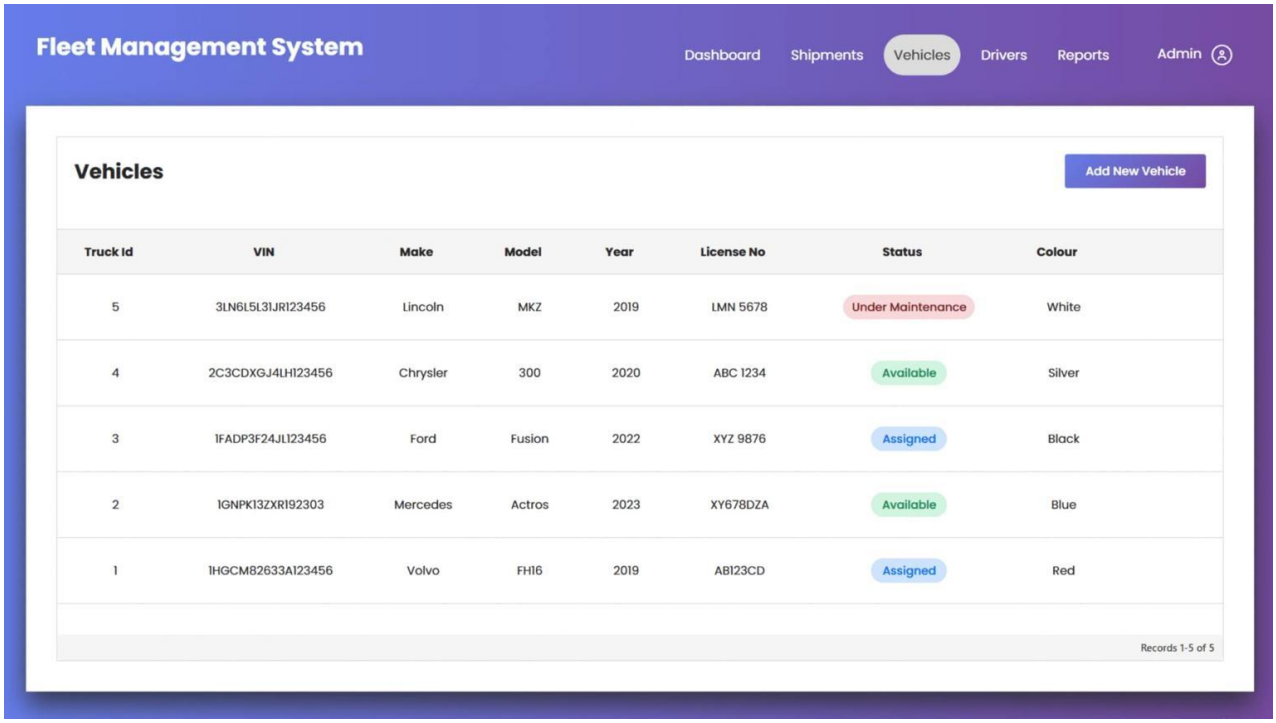
Screenshot 2 — Dashboard

Displays the number of imported transport records and navigation links.



Screenshot 3 — Spreadsheet upload

Upload a CSV or Excel file and receive the import result.



Screenshot 4 — Imported records

Displays vehicle numbers, drivers, routes, and distances.

These are visual references; capture actual screenshots from your running application for your project documentation.

11. Project report content

- Abstract: This project automates the import of transport information from spreadsheets into a database.
- Existing system: Transport details are entered manually, which can cause duplicate entries and errors.
- Proposed system: Spreadsheet uploads, validation, duplicate detection, and automatic database storage.
- Advantages: Saves time, reduces manual work, improves data consistency, and makes records easier to access.
- Future enhancement: Add user registration, export to Excel, route reports, vehicle maintenance records, and an import history page.
- Conclusion: The Transport Data Import Management System provides a simple way to manage transport records using spreadsheet-based data import.

One important clarification: If your college gave you this exact topic from a ServiceNow practical or project, tell me. In ServiceNow, Import Data Using Transform Maps (Spreadsheets) means uploading an Excel/CSV file into an Import Set table and using a Transform Map to map its columns to a target table. That is a different project from the Python application above.